Module 3 – Privacy & Communication

Work Completed and Contribution Report

Federated RLHF / Privacy-Preserving Federated Learning Project

1. Purpose of Module 3

Module 3 is the Privacy & Communication module. Its purpose is to ensure that clients do not send raw user data to the central server. Instead, clients produce model updates (eventually LoRA adapter updates), and this module prepares those updates for efficient and privacy-aware transfer and aggregation.

According to the project specification, Module 3 is responsible for preparing model updates, compressing and serializing updates, prototype-level encryption, secure aggregation, and communication optimization. The specified implementation files are aggregation.py, compression.py, communication.py, and serializer.py.

2. Importance of Module 3 in the Overall System

The overall federated workflow is: the server distributes a global model, clients train locally, clients produce LoRA adapter updates, the privacy layer compresses and protects those updates, and the server aggregates the received updates to create the next global model.

Therefore, Module 3 forms the boundary between local model training and server-side aggregation. Its importance is not to train the LLM, but to make the model-update exchange more private, smaller, and suitable for communication.

Project Part	Module 3 Role	Importance
Local LoRA Training	Receives the resulting adapter/model update	Defines the input to the privacy/communication pipeline
Serialization	Converts the update into transferable bytes	Provides a standard representation for transmission
Compression	Reduces the size of serialized updates	Reduces communication cost and bandwidth
Privacy	Applies privacy mechanisms such as DP	Reduces the risk of exposing information from updates
Secure Aggregation	Combines updates without exposing individual contributions	Protects client-level updates during aggregation
Communication	Transfers prepared updates between client and server	Connects local training to federated aggregation

3. Work Completed So Far

Component	Status	Work Completed
Python 3.11 development environment	Completed	Installed Python 3.11.15 using pyenv and created a project-specific .venv environment.
Project/package setup	Completed	Created the privacy/ and tests/ structure, added __init__.py, pytest.ini, and pyproject.toml, and installed the project in editable

		mode.
serializer.py	Completed	Implemented serialize_update() and deserialize_update() using pickle.
Serializer testing	Completed	Created a pytest test using a simulated multi-parameter LoRA-style update and verified serialization/deserialization with 1 passing test.
Serializer demonstration	Completed	Created a demonstration script that displays parameter shapes, serialized type/size, restored parameters, and verification results.
compression.py	Completed – basic prototype	Implemented gzip compression/decompression functions.
Compression testing	Completed – basic prototype	Created a pytest test verifying that compressed data can be decompressed back to the original bytes.
Compression demonstration	Completed – basic prototype	Created a demonstration script to show original size, compressed size, compression ratio, and successful restoration.

4. Detailed Explanation of Completed Work

4.1 Python 3.11 and Virtual Environment

A dedicated Python 3.11.15 environment was established because the team's eventual federated framework is based on Python 3.11. The project now uses a .venv environment, keeping dependencies isolated from the system Python.

4.2 Project Structure and Testing Setup

The independent Module 3 workspace was organized so implementation and tests are separated:

```
EDI/
├── privacy/
│   ├── __init__.py
│   ├── serializer.py
│   ├── compression.py
│   ├── communication.py
│   └── aggregation.py
├── tests/
│   ├── test_serializer.py
│   ├── test_compression.py
│   └── demonstration scripts
├── pytest.ini
├── pyproject.toml
└── .venv/
```

4.3 Serialization

Serialization converts a Python model-update object into bytes so it can be processed for communication. The implementation uses Python's pickle module.

Implemented functions:

> ***serialize_update(update)***
>
> ***deserialize_update(data)***

The first test used a simple NumPy matrix to verify the mechanism. The test was then improved to simulate a more realistic LoRA-style update represented as a dictionary containing multiple parameter matrices, for example layer1.lora_A, layer1.lora_B, layer2.lora_A, and layer2.lora_B.

The serializer test verifies that the serialized result is bytes and that every parameter is identical after deserialization. The test currently passes successfully with pytest.

4.4 Compression

Compression operates after serialization. The serialized bytes are compressed before communication and decompressed on the receiving side.

Implemented functions:

> ***compress_update(data)***
>
> ***decompress_update(data)***

gzip is currently used as the prototype compression method. A test verifies that compression followed by decompression returns exactly the original byte sequence. A demonstration script also reports original size, compressed size, compression ratio, and restoration status.

Important limitation: the current compression demonstration uses simple sample bytes. It is a prototype validation of the mechanism, not yet the final benchmark using real LoRA adapter weights.

5. Current Module 3 Pipeline

The implemented portion currently represents the beginning of the intended Module 3 pipeline:

> ***LoRA-style / model update → Serialization → Compression → [Privacy Protection] → [Communication] → [Secure Aggregation] → Server/FedAvg***

6. What Is NOT Completed Yet

- Differential Privacy implementation and experiments.
- Secure Aggregation prototype.
- Communication layer for sending and receiving updates.
- Prototype-level encryption, if retained as a separate implementation step.
- Integration with the group's Flower/Ray federated client-server framework.

- Integration with the actual PyTorch/LoRA adapter output from the LLM Training module.
- Realistic communication-cost and bandwidth measurements using actual adapter updates.
- End-to-end testing of the complete privacy/communication pipeline.

7. Recommended Next Development Order

1. Improve compression experiments using larger, LoRA-style parameter updates.

2. Implement Differential Privacy as a simplified prototype.

3. Implement and test Secure Aggregation using multiple simulated clients.

4. Implement communication interfaces for sending and receiving prepared updates.

5. Define clear interfaces between Module 2's LoRA output, Module 3, and Module 1's server.

6. Integrate Module 3 with Flower.

7. Run an end-to-end federated round and measure communication/privacy behavior.

8. What Can Be Demonstrated to the Project Guide

- A working Python 3.11 project environment with isolated dependencies.
- A clean privacy/ and tests/ module structure.
- Working serialization and deserialization of a simulated multi-parameter LoRA-style update.
- Automated pytest validation showing the serializer test passes.
- Working gzip compression/decompression.
- A compression demonstration that measures original size, compressed size, compression ratio, and data restoration.
- A clear roadmap showing how these components will connect to Differential Privacy, Secure Aggregation, Communication, and later Flower integration.

9. Current Contribution Summary

At the current stage, Module 3 has progressed from environment setup to a functioning prototype for two core update-processing components: serialization and compression. The work demonstrates that model-update data can be represented as bytes, restored correctly, compressed, and decompressed without data loss. The remaining work is the privacy and communication portion: Differential Privacy, Secure Aggregation, communication, encryption at prototype level, and integration with the team's Flower-based federated system.

10. Reference to Project Specification

The project specification assigns Member 3 to Privacy & Communication. It states that the module should prepare, compress, serialize, and protect model updates, support secure aggregation and communication optimization, and use simplified prototype implementations where appropriate. It also specifies the four core implementation files: aggregation.py, compression.py, communication.py, and serializer.py.